

FOCUS 2026 年秋季招新控制组第二阶段考核

考核周期：9 月 20 日 - 10 月 6 日

一、考核内容一：基础政策与原则

请认真阅读并理解以下基础政策和原则，确保不违背本次招新的根本精神。

1.1 考核目标

满足以下要求之一即可：

1. 选拔出具有**自驱力**、对机器人方向怀有足够热爱，愿意在此领域投入大量时间和精力的人才（而非过于功利、唯加分或评奖评优论者）。
2. 选拔出**敢于尝试**、不墨守成规，愿意在大学阶段积极探索技术、创业或科研方向的人才。
3. 选拔出具备**组织、表达、团结与领导能力**，能够带动身边队友共同进步、协调项目进度的人才。
4. 选拔出**深造意愿强烈**，对攻读博士、从事科研工作充满热情的人才。

1.2 出题原则

1. 假定参与考核的同学为有志于往机器人方向发展的**大学一年级新生**。考核内容旨在补全机器人从业者及机器人专业学生所必需且共通的**底层能力与视野**，帮助其对未来的机器人学习路线形成初步规划。
2. 假定学生具备**科学上网能力**，并能够使用 **AI 工具**（若尚不具备，需主动向周围同学请教学习）。
3. **不假定**学生具备中等及以上的基础能力（即不假定其在高中参加过机器人或信息学相关竞赛，或接触过任何编程语言）。

1.3 AI 使用政策（AI Policy）

1. 本说明适用于本次考核**全程及全部内容**。
2. 你可以、也应当将 AI 视为**能力远超于你的协作者或老师**，应经常向其请教问题，将其作为你最得力的帮手。
3. 但在此次考核中，你**绝不能将自己定位为产品经理、将 AI 定位为工程师来直接交付结果**，否则你将无法从中获得真正的成长。
4. 请不要完全依赖 Codex、Claude Code 等 AI 代码工具生成代码，而自身对基础细节一无所知。面试时将考察原理、应用等方面的内容。
5. 对于环境配置等问题，你可以尽情地使用 AI。
6. 所有人所使用的 AI 能力相同或接近，**真正区分你与他人的，是基本功**。

1.4 本套试题使用方式

1. 以下部分分为**强化学习、嵌入式基础、ROS 2 机器人仿真与视觉、电路基础四大部分**，每个部分均有若干个 Task。四大部分分别聚焦**算法、嵌入式控制、机器人软件与视觉、电路设计四大控制组细分岗位**。对于基础薄弱的大一同学，建议根据自己的爱好，专精一大项完成。对于大二的同学，建议选择两大项及以上**完成。

2. 当然，做哪部分的题目不代表你以后在项目中要从事哪种特定类型的工作，我们欢迎同学们拓展自己兴趣的**广度与深度**，甚至成为**全栈工程师**。

二、强化学习入门

Task 1：阅读

任务目标

了解 PyTorch 在神经网络中的基本用法和一般训练流程，了解基本的概率论知识。

阅读材料

- 引用自 CS285 的两份 Note（已随考核材料提供，见：[26秋招新题目控制组阶段二\强化学习\阅读材料](#)）
- 其他学习资料：
 - [PyTorch深度学习快速入门教程（绝对通俗易懂！）【小土堆】](#)

任务注解

- 多看两遍是正常的。
- 请不要在这两个阅读任务上花费过长时间，**建议控制在 12 小时以内**。
- 此任务是为后续的 RL 学习任务打基础。

Task 2：策略梯度

任务目标

大致理解策略梯度的原理并完成 [强化学习/Lab](#)，**不必深究数学原理**。

注：你可以选择更深入地了解，此为**加分项**。

推荐学习资料

- CS285 2026 相关 PPT（见：[26秋招新题目控制组阶段二\强化学习\task2_lab\学习资料](#)）
- B站教学视频：
 - [【英文原声 | 中英字幕】2026年春季最新！伯克利分校 CS 185/285 深度强化学习、决策与控制](#)
- 任何强化学习相关教程，此处推荐：
 1. [【强化学习的数学原理】课程：从零开始到透彻理解（完结）](#)
 2. [【王树森】深度强化学习\(DRL\)](#)

Lab 任务

- **实验指导书：**强化学习/Lab（已随考核材料提供）
- **按照指导书完成全部内容**

任务注解

1. 数学原理看不懂是正常的，感性理解即可。
2. **Lab 环境建议：**请严格参照 README 中的说明进行配置。
3. 指导书中的"提交部分"不需要处理，只需按规定方式提交到团队。
4. 完成代码编写和训练后，可以尝试调参或使用额外技巧提升模型效果。
5. **建议耗时不超过 10 天。**
6. 关于教程，Sergey Levine 确实是具身领域的大牛，但是可惜他讲课我经常听不懂；赵老师的课程非常易懂，但不够深入；王树森的课有点杂，而且不如上面两者的理论性，因此自己斟酌怎么学习吧。
7. 本题目编码难度不高，编码主要是为了解算法，因此不允许使用AI生成代码，必须自己写。

Report 要求（这是我们评分的主要依据，请重视）

1. 任务完成后，移除全部的（但不是删除，包括 WandB 也不能删除，我们可能会面试的时候查看）权重，只保留源码，以及在 强化学习 目录下面命名一份 Report.pdf 文件，然后提交。
2. 关于 Report，你可以发挥自己的主观能动性来优化表达，但是你至少应当完成我的以下要求（可以从 WandB 截图）：
 - 每一个任务（包括同一个仿真任务的不同参数设置，各自独立计算）的 **Loss 曲线**
 - 每一个任务的 **Return 曲线**（std 即可）
 - **参数设置说明**（全部超参数，可以在 WandB 里面找到）

说明：对于 RL lab 里面的所有实验，均需要在 Report 里面说明（附上超参数）+ 截图（可以使用 WandB 里面的图片）。

校验方法：完成题目，用时10小时左右

三、嵌入式基础

任务目标

学习嵌入式开发的工具链、了解嵌入式开发的完整流程，学习基础接线方法。

使用 Arduino IDE 和 ESP32 Arduino Core 完成从 GPIO、PWM 到通信与状态机的实践，理解嵌入式开发的基本概念并完成任务。

本篇为 ESP32 Arduino 嵌入式开发方向专属笔试题，聚焦 ESP32 硬件原理、Arduino 固件开发、外设控制、物联网应用等核心考点，全面检验你的嵌入式开发能力。

任务使用 Arduino 语言（C/C++）和 Arduino Core for ESP32 提供的常用 API，先快速实现功能，再逐步理解底层原理。

本部分提供 **两个开发方向，二选一**完成即可：**方向一：ESP32 Arduino**（Arduino IDE，见上文）；**方向二：ESP-IDF**（乐鑫官方框架，更贴近底层驱动，见下文‘方向二：ESP-IDF’）。其中，**大二同学限定只能选择方向二完成**。

方向一：ESP-Arduino

ESP32 是由乐鑫信息科技推出的高集成度、低功耗 32 位物联网系统级芯片（SoC），集成 2.4GHz Wi-Fi 与蓝牙双模通信模块，

拥有丰富的外设接口与完善的开发生态，兼具高性能与低功耗特性，广泛应用于智能硬件、物联网终端、嵌入式控制等场景，

是嵌入式物联网开发中极具代表性的主流平台。

推荐同步观看以下入门教程：

- 视频教程：[2023年最新 ESP32 Arduino 教程（持续更新中）](#)
- 文档教程：[ESP32 Arduino 系列教程](#)

题目介绍

点灯是一个嵌入式工程师的第一步，本次笔试题将从点灯出发，一步步围绕“点灯”，从PWM、基本通讯协议、无线通讯协议、还有简单的深度学习应用，做一盏多模态控制的智能灯，并一步步加深我们的技术栈。

物资需要

1. 核心硬件：

- 核心开发板：ESP32-S3开发板。
- 灯：ESP32开发板上带RGB三色灯，点亮板载三色灯。
- 显示屏：一块与 ESP32-S3 兼容的 TFT 彩色屏幕，用于完成 TASK6（仅大二同学需要）；具体型号自行选购的硬件确定。

2. 辅助工具：USB数据线（Type-C）。面包板、杜邦线等（仅大二同学需要，用于做TASK6） （我们已在群中发布需要的硬件，若仍未购买请尽快）

参考购买链接：

[ESP32-S3-N16R8（带天线座）（焊接）](#)

任务介绍

本次笔试题从易到难分为**6个TASK**，请各位按顺序逐步完成。**ATTENTION：**

- 基础要求：只要求为每个Task编写独立demo验证功能。**但是：**可最终提交**整合版工程**，兼容所有已完成Task的点灯方式（即支持多模态控制并存，无功能冲突），则是加分项。整合版工程建议大二同学做到。
- github仓库请写一篇简要的README.md介绍你的作品。
- 所布置任务完成程度（进度、效果）会影响最终得分。
- 其中TASK6是扩展题，大二同学需完成，大一同学不做要求。

以下是各个**任务说明**：

- TASK1：物资准备、环境安装及验证：**

安装 Arduino IDE，配置ESP32 开发环境。

可参考上方视频与文档教程完成驱动、开发板和串口配置。

配置好环境后，连接硬件，使用 Arduino API，编写并上传第一个程序，点亮你的 LED。

- **TASK2: 改进你的灯为呼吸灯:**

PWM（脉冲宽度调制）是嵌入式开发中实现“模拟量调节”的核心技术——它通过控制高低电平的占空比，将数字信号模拟为连续变化的电压 / 电流，这也是调节 LED 亮度的核心原理。

本任务中，你需要先理解 PWM 的工作机制（占空比、频率与输出电平的关系），再使用 Arduino ESP32 的 LEDC/PWM API，

将普通的亮灭灯改为**呼吸灯**：实现 LED 亮度在设定范围内平滑渐变（从暗到亮、再从亮到暗循环）。

- **TASK3: 实现有线协议点灯:**

串口（UART）协议是嵌入式设备中最基础、最通用的有线通信方式之一。

你需要掌握波特率、数据位、停止位、校验位等基本概念，并配置串口、收发数据，最终实现**通过串口指令控制 ESP32 开发板完成 LED 亮灭与亮度控制**。

指令格式自行设计，但需在 README 中说明。

- **TASK4: 实现无线协议点灯:**

ESP32 能风靡全球的原因离不开它原生的无线通信功能。

使用 Arduino ESP32 提供的 Wi-Fi 与 Bluetooth/BLE 库，实现无线联动控制灯的明灭与亮暗。

1. 蓝牙：学习 BLE 的 GAP/GATT 基础，使用 Arduino 的 BLE 库创建服务和特征，通过手机发送指令控制 LED。蓝牙配网可作为拓展，不作必备要求。
2. 网络：了解 IP、端口、Socket 等基本概念，使用 `WiFi.h` 连接路由器并完成简单的 TCP/HTTP 通信；有余力可使用 PubSubClient 等库接入 MQTT，实现远程控制。

- **TASK5: 状态机编程**

在 Arduino 工程中，循环函数 `loop()` 会持续处理传感器、串口和网络事件。将这些事件组织成有限状态机，可以让程序在功能增加后仍保持清晰、可维护。

状态机（有限状态机 FSM）将设备逻辑拆解为若干离散状态（如“熄灭”“常亮”“呼吸”“闪烁”“联网中”“已联网”），并定义串口指令、按键、Wi-Fi 状态和定时器等事件触发状态切换的规则。

建议使用 `millis()` 实现非阻塞定时，避免在 `loop()` 中长时间 `delay()` 阻塞其他功能。

你需要实现状态机的编程，将 LED 的“熄灭 / 常亮 / 呼吸 / 闪烁”等作为核心状态，把“串口指令、网络连接、MQTT 指令、定时器事件”等作为状态切换的触发事件，自行设计并实现一套基于状态机的灯控逻辑。

- **TASK6 (扩展题) : 使用 TFT 屏幕实现创意功能**

TFT 屏幕能够显示文字、图形、图片和动画，是嵌入式设备与用户交互的重要窗口。本任务要求你使用 ESP32-S3 驱动 TFT 屏幕，并在此基础上自行构思、设计和实现一个具有完整效果的创意功能。

创意方向不限。例如：制作联网时钟或天气界面、播放动画或动态视频、绘制实时数据曲线、制作电子相册、小游戏、设备状态仪表盘，或将屏幕与前面任务中的灯光、串口、蓝牙和网络功能联动。以上仅为示例，我们更期待你实现自己的想法。

基础要求:

1. 根据所用屏幕的型号和通信接口正确完成接线，使 ESP32-S3 能够稳定驱动 TFT 屏幕。
2. 屏幕应至少能够正确显示文字、图形或图片，并在此基础上实现一项完整的创意功能，而非仅运行图形库自带的示例程序。
3. 动态内容应连续、稳定地运行；程序不应出现明显卡死、频繁闪屏或异常重启。

4. 在 README 中说明屏幕型号、引脚连接、所用图形库、功能设计和使用方法，并附上能够展示实际运行效果的图片或视频。

建议探索：

- 了解所用 TFT 屏幕的分辨率、色彩格式及 SPI 或其他通信方式的基本原理。
- 思考图片、字体或视频帧如何存储和读取，并合理利用 ESP32-S3-N16R8 的 Flash 与 PSRAM。
- 对动态界面合理控制刷新区域和刷新频率，尝试减少闪烁、撕裂与不必要的全屏刷新。
- 尝试让屏幕与 Wi-Fi、BLE、串口、按键或已有灯控系统产生交互。

本任务不以界面复杂程度作为唯一评价标准。创意的完整性、实际运行效果、对实现原理的理解，以及解决调试问题的过程同样重要。

参考资料

1. [ESP32 Arduino 视频教程 \(Bilibili\)](#)
2. [ESP32 Arduino 系列文档](#)
3. [Arduino ESP32 Core 官方文档](#)
4. [ESP32-CAM中文资料](#)
5. [PubSubClient \(MQTT Arduino 库\)](#)

本部分提交要求

1. 提交完整、可构建和可运行的工程。（可以分块提交）上传到你的 GitHub 仓库。
2. 在 README 中写明：开发环境、依赖，以及各任务完成情况。说明主要设计思路、关键参数、调试过程及尚未解决的问题。即使部分功能未完成，也应保留有效尝试和分析。
3. 使用截图或视频展示各部分的实际运行效果。建议在提交文件夹中特定位置放置这些截图、视频，并进行清晰的重命名。

方向二：ESP-IDF（乐鑫官方框架，与方向一 Arduino 二选一，两个方向都完成视为加分项）

说明

：本节为嵌入式基础的

ESP-IDF 方向

，与上文“使用 Arduino IDE 和 ESP32 Arduino Core”的方向

二选一

完成即可；两个方向都完成视为加分项。

选用 ESP-IDF（乐鑫官方框架）而非 Arduino，是为了让你直接接触协议栈与驱动更底层的东西

——没有公司会认为熟练使用 Arduino IDE 的人就是优秀的嵌入式工程师。

任务目标

学习嵌入式开发的工具链与完整流程，使用 **ESP-IDF**（而非 Arduino）依次完成：GPIO 点灯 → PWM 呼吸灯 → 串口协议点灯 → 无线协议点灯（Wi-Fi / 蓝牙二选一，推荐 Wi-Fi）→ **直流电机控制**（无级调速 → 编码器测速 → PID 闭环）→ **EEZ Studio + LVGL 触摸屏控制**，理解 ESP32 底层驱动、FreeRTOS 任务模型与图形界面开发，最终做一个可通过屏幕、串口、网络 / 蓝牙多通道控制的智能设

备。

推荐同步观看 / 阅读：

- 视频教程（正点原子·IDF 版）：[【正点原子】ESP32 快速入门开发秘籍（IDF 版）【真人出境】手把手带你快速入门 ESP32 嵌入式 单片机（高清带字幕）](#)
- 官方文档：[ESP-IDF 编程指南](#)
- 文档教程：[ESP-IDF 入门指南](#)

环境要求（版本号务必统一）

ATTENTION
：以下版本为命题机统一环境。请严格按此安装，版本不符可能导致编译不通过或 API 不一致。

软件 / 组件	版本	说明
ESP-IDF	v5.5.4 (不高于 5.5)	官方开发框架，用更高版可能编译不通过
芯片目标	ESP32-S3	<code>idf.py set-target esp32s3</code>
lvgl/lvgl (LVGL)	9.2.0	图形库，TASK6 使用；在 <code>main/idf_component.yml</code> 中声明 <code>lvgl/lvgl: ^9.2.0</code>
espressif/esp_lvgl_port	2.8.0	LVGL ↔ ESP-LCD 桥接组件，TASK6 使用；声明 <code>espressif/esp_lvgl_port: ^2.8.0</code>
espressif/esp_lcd_touch_ft5x06	1.1.0	FT6336U/FT5x06 触摸驱动，TASK6 使用；声明 <code>espressif/esp_lcd_touch_ft5x06: ==1.1.0</code>
EEZ Studio	v0.29.0	GUI 可视化设计工具（导出 LVGL 代码），TASK6 使用； 官方下载
蓝牙模块 HC-04D（外接）	无需额外软件	SPP 2.1 + BLE 4.0 双模，UART 透传，TASK4 蓝牙点灯使用（外接方案）
esp_wifi / esp_http_server	ESP-IDF v5.5.4 自带	TASK4 Wi-Fi 点灯使用
LEDC / PCNT	ESP-IDF v5.5.4 自带	LEDC：TASK2 呼吸灯 / TASK5 电机调速；PCNT：TASK5 编码器测速

开发方式：**Windows + ESP-IDF**（命题机环境为 Windows）。

使用乐鑫官方 **ESP-IDF Windows 安装器**（在线 / 离线安装包均可）一键安装工具链；安装完成后通过桌面快捷方式 "ESP-IDF PowerShell / ESP-IDF CMD" 打开已配置好环境变量的终端，即可运行 `idf.py` 命令。

物资需要

1. 核心硬件：

1. 核心开发板：**ESP32-S3-N16R8**（16MB Flash / 8MB PSRAM，带排针焊接版）。
参考购买链接：[ESP32-S3-N16R8（带天线座）（焊接）](#)
2. 灯：板载 RGB 灯，或外接 5mm LED（方便展示效果，建议外接）。
3. 直流电机：**5V 直流 TT 马达**（普通版即可，TASK5）；**带编码器版本**（拓展题用，务必带编码器，如 [松甲 霍尔编码器 TT 马达 AB 相 智能小车电机 PID 测速](#)，可当普通直流电机使用）。
4. 电机驱动模块：**L9110S**（最简单，两路 GPIO 控制 IN1/IN2）或 **TB6612**（PWM + 方向引脚，更推荐，可同时驱动两路），参考购买：[HG7881 / L9110S 两 / 四路直流电机驱动板（智能小车）](#) · [TB6612FNG / DRV8833 电机驱动模块（直流电机 / 自平衡小车）](#)。
5. 显示屏（TASK6 需要）：**ST7789V 240×320 SPI 接口 + FT6336U 电容触摸**，参考购买：[鸿讯 2.4 寸屏幕模块 240×320 RGB 电阻 / 电容触摸 18pin 插接 SPI 3/4 线](#)。（记得和转接版一起买）
6. **辅助工具**：Type-C 数据线（能传数据的）、面包板、杜邦线若干、**5V 电源**（给电机驱动供电，与开发板共地），参考购买：[5V 电源适配器 + 充电器](#)。
7. **软件**：ESP-IDF v5.5.4、EEZ Studio v0.29.0（TASK6 用）。
8. **蓝牙模块**（TASK4 用）：**HC-04D 双模蓝牙模块**（SPP 2.1 + BLE 4.0，UART 透传，带底板焊排针），参考购买：[HC-04D 蓝牙模块 双模 SPP2.1 + BLE4.0 无线串口通信模块](#)；
9. **手机**：安装微信（TASK4 蓝牙点灯用 "蓝牙串口 - 江协科技" 小程序）。

任务介绍

本次笔试题从易到难分为 **6 个 TASK**，请按顺序逐步完成。**ATTENTION：**

- 基础要求：每个 Task 编写独立 demo 验证功能。**但：**可最终提交**整合版工程**，兼容所有已完成 Task 的控制方式（多模态控制并存，无功能冲突），这是加分项，建议大二同学做到。
- 编程时请有基本的**多任务 / 线程**概念，不要用 `vTaskDelay` 把主流程写死。
- github 仓库请写一篇简要的 README.md 介绍你的作品（环境与版本、指令格式、引脚、完成进度、运行效果截图 / 视频）。
- 所布置任务完成程度（进度、效果）会影响最终得分。
- 其中 TASK5 的拓展部分，**大二同学需完成**，大一同学不做要求。TASK6 是选做内容，不做硬性要求，但是加分项。

以下是各个任务说明：

TASK1：物资准备、环境安装及验证 —— GPIO 点灯

配置好 ESP-IDF v5.5.4 环境（见上方环境要求），连接硬件，使用 ESP-IDF 的 **GPIO 驱动**点亮你的 LED，并在串口 monitor 中打印日志确认。

需要掌握的基础概念：

- GPIO 输出模式：推挽输出 / 开漏输出的区别；
- 上拉 / 下拉电阻的作用；

- `gpio_set_direction()`、`gpio_set_level()` 等驱动 API 的用法；
- 延时与周期性翻转电平，实现 LED 闪烁。

TASK2: 改进你的灯为呼吸灯 —— LEDC PWM

PWM（脉冲宽度调制）是嵌入式开发中实现“模拟量调节”的核心技术——它通过控制高低电平的**占空比**，将数字信号模拟为连续变化的电压/电流，这也是调节 LED 亮度的核心原理。

本任务中，你需要先理解 PWM 的工作机制（**频率、分辨率、占空比**三者的关系），再基于 ESP-IDF 的 **LEDC 驱动接口**（`ledc_timer_config` + `ledc_channel_config`），将普通的亮灭灯改造为**呼吸灯**：实现 LED 亮度在设定范围内平滑渐变（从暗到亮、再从亮到暗循环），直观体会 PWM 在硬件控制中的实际应用。

注：TASK5 电机调速用的也是 LEDC，两者原理完全一致，只是频率和驱动对象不同。

TASK3: 实现有线协议点灯 —— UART 串口

串口（UART）协议是嵌入式设备中最基础、最通用的有线通信方式之一。你需要先掌握串口协议的核心规则（**波特率、数据位、停止位、校验位**的作用，以及数据收发的底层逻辑），并基于 ESP-IDF 的 **UART 驱动**配置串口参数、编写收发逻辑，最终实现**通过串口指令控制 LED 亮灭与亮度**。

工程要求：

- 使用 `uart_driver_install()` 配置串口（建议 115200, 8N1）；
- 采用**中断接收 + 环形缓冲**，避免轮询占用 CPU；
- 指令格式自行设计，但需在 README 中说明。示例参考：`ON`（亮）、`OFF`（灭）、`SPEED 50`（设置亮度 0~100）；
- 指令解析具备基本容错能力：参数越界、格式错误时返回错误提示（如 `ERROR`）。

TASK4: 实现无线协议点灯 —— Wi-Fi / 蓝牙（二选一，都做加分）

ESP32 风靡全球的原因离不开它原生的无线通信功能。以下 **Wi-Fi 与蓝牙任选其一**，两条都完成按加分项处理：

1. **Wi-Fi 点灯**：了解网络基本概念（IP、端口、Socket 等）。实现 Wi-Fi **STA 模式**连接路由器，用 `esp_http_server` 在开发板上起一个 HTTP 服务器，手机/电脑浏览器访问 `http://<开发板IP>` 打开一个**点灯网页**，通过网页按钮控制 LED 亮灭与亮度。
 - 要求：只需实现 HTTP 方式；（可借助 AI 写网页并移植）
 - 页面可以内嵌到固件（`CMakeLists.txt` 用 `EMBED_FILES` 嵌入 HTML），也可以直接在网页里写 JS 调接口；
 - 在 README 中说明你的 IP 获取方式（串口打印 / DHCP 静态分配）。
2. **蓝牙点灯（外接 HC-04D 模块，用江协微信小程序）**：采用**外接蓝牙模块方案**（与江协科技/江科大教程一致）。HC-04D 为 **SPP 2.1 + BLE 4.0 双模**无线串口透传模块，开发板只需通过 **UART**（复用 TASK3 的串口驱动）与模块通信，模块负责无线透传。
 - 接线：HC-04D 的 TXD → ESP32 的 RXD、RXD → TXD、VCC → 3.3V/5V、GND 共地（模块带底板）；建议接 **UART1**（UART0 默认是日志/烧录口，避免冲突）；
 - 配置：上电后可用 **AT 指令**查询/修改模块参数（如蓝牙名称、波特率），先在电脑串口助手验证模块收发；
 - 手机端：**微信搜索小程序“蓝牙串口 - 江协科技”**，扫描并连接你的蓝牙模块，通过小程序收发指令控制 LED 亮灭与亮度（指令格式与 TASK3 串口一致，自行设计并写入 README）；

- 推荐先看江协科技（江科大）的蓝牙串口 / 无线通信教学视频（外接模块 + 手机收发），理解串口透传与蓝牙数据链路。

TASK5: 直流电机控制 —— 无级调速 · 编码器测速 · PID 闭环

本任务核心考点：GPIO 驱动 → PWM 无级调速 → 串口控制 → 编码器测速 → PID 闭环

基础任务：控制普通直流电机（启停 + 无级调速 + 方向 + 串口控制）；拓展任务（大二）：控制编码器电机，实现转速测量与 PID 速度闭环。

一、基础任务：普通直流电机控制

1. 硬件接线与供电

- 驱动模块二选一：
 - **L9110S**（最简）：IN1 / IN2 接两个 GPIO；启停 / 正反转用 GPIO 电平（IN1=1、IN2=0 正转，反之反转，同电平停止），PWM 调速时 IN1 接 PWM、IN2 接方向电平（或互换）；
 - **TB6612**（推荐，可扩展双路）：PWMA 接一个 GPIO（PWM 调速），AIN1 / AIN2 接两个 GPIO（方向）。
- 供电：5V 电源单独给驱动模块供电，开发板与驱动**共地**（GND 相连）；严禁用开发板 3.3V 直驱电机。

1. GPIO 驱动电机启停

- 配置 2 个 GPIO 为**推挽输出（无上下拉）**，用 `gpio_set_direction()` + `gpio_set_level()` 控制 IN1/IN2（或 AIN1/AIN2），实现电机启停与正反转。
- 验证标准：电机稳定启停、无卡顿 / 异响，驱动芯片与电机无异常发热。

1. PWM 无级调速

- 用 **LEDC**（`ledc_timer_config()` + `ledc_channel_config()`，思路与 TASK2 呼吸灯完全一致，只是频率与负载不同）输出 PWM 到驱动模块，理解 **占空比 → 等效电压 → 电机转速** 的链路。
- 相比 GPIO 直接驱动，PWM 调速能避免电机瞬间大电流堵转，提升控制精度。
- **实现转速自动渐变：慢 → 中 → 快 → 中 → 慢 循环**（禁止用阻塞延时写死）。参考做法：FreeRTOS 任务中按档位周期更新占空比（`ledc_set_duty()` + `ledc_update_duty()`），或使用 LEDC fade 渐变接口。

1. 串口控制电机

- 沿用 TASK3 的 UART **中断接收 + 环形缓冲**（避免轮询占用 CPU），指令格式自行设计并在 README 说明，示例：

指令	说明
ON / OFF	启动 / 停止电机（停止时占空比置 0）
SPEED 50	设置转速（参数 0~100，对应占空比 0%~100%）
DIR F / DIR R	正转 / 反转

- 容错要求：参数越界、指令格式错误时返回 `ERROR`。

二、拓展任务：编码器电机 + PID 速度闭环

学习参考：

[【PID 入门教程 - 电机控制 倒立摆 全程手把手打代码调试】 \(B 站\)](#)

[电机控制进阶 ——PID 速度控制 \(CSDN, 含参数整定实验\)](#)

(
[配套演示视频](#)

) ·
[PID 算法讲解 + PID 电机闭环控制介绍 \(CSDN\)](#)

1. 编码器测速

- 换用**带编码器的 TT 马达**（购买时务必与客服确认带编码器）。理解编码器测速原理：电机每旋转一周，编码器输出固定数量的脉冲，统计单位时间内的脉冲数即可换算实际转速：

1 | $\text{RPM} = \text{采样周期内脉冲数} \div \text{每转脉冲数} \div \text{采样时间 (分钟)}$

示例：编码器每转 20 个脉冲，采样 100ms 数到 50 个脉冲，则 $\text{RPM} = 50 \div 20 \div (0.1/60) = 1500$ 。

- 使用 ESP32-S3 内置 **PCNT 外设**（硬件脉冲计数，等价于 STM32 的 "定时器编码器模式"：倍频更高、采样更精确、CPU 占用更低；GPIO 外部中断法也可尝试，但有高频中断导致 CPU 占用过高的风险）。用 `pcnt_unit_config()` / `pcnt_channel_config()` 配置计数单元与输入滤波，定时 `pcnt_unit_get_count()` 读取计数并 `pcnt_unit_clear_count()` 清零。
- 在 README 记录：编码器线数 / 圈、倍频方式、脉冲计数与转速换算公式。

2. PID 精准转速闭环

- 理解 PID 三要素：**P（比例）**——按偏差成比例输出，快速消除偏差；**I（积分）**——累积历史偏差，消除稳态误差；**D（微分）**——预测偏差变化趋势，抑制超调，但对噪声敏感（速度环可只用 PI）。
 - 控制回路：`目标转速 - 实际转速 (PCNT 测得) → PID 计算 → 更新 PWM 占空比`，控制周期建议 10~50ms。
 - **参数整定方法（先 P 后 I）：**
 1. 令 $I=0$ 、 $D=0$ ，P 从小到大逐步加大，观察转速阶跃响应，取 "开始明显震荡前 P 值的 50%~60%" 作为 P；
 2. 加入 I 并逐步增大，消除稳态误差（若出现低频振荡则减小 I）；
 3. 记录每组 P/I 对响应速度、超调量、稳态误差的影响，把整定过程写进 README。
 - **验证方法：**串口发送 `SET_RPM 250`，串口持续实时输出 `目标转速: 250 → 实际转速: xxx`，观察转速从变化到稳定的快慢（阶跃响应）、稳态误差与超调情况。
3. **README 要求：**记录编码器分辨率、测速公式、PID 参数 (P/I 值) 与整定过程、调试中遇到的问题与解决方法。

TASK6 (选做题)：EEZ Studio + LVGL 触摸屏控制

TFT 触摸屏是嵌入式设备与用户交互的重要窗口。本任务要求你用 **EEZ Studio 可视化设计界面**，配合 **LVGL v9** 在 ESP32-S3 上实现一块**可触摸控制的屏幕**，把前面 TASK1~TASK5 的灯光与电机控制收拢到屏幕上来。

完成本任务所需具备的能力（前置要求）：

- **SPI 总线** (LCD 用)：理解 SPI 时序 (SCK / MOSI / CS / DC 信号) 与全双工收发原理，会用 `esp_lcd` + `esp_lcd_new_panel_io_spi()` 初始化 ST7789V 面板；

- **I2C 总线**（触摸用）：理解 I2C 时序（SCL / SDA）、从机地址与寄存器读写，会用 `esp_lcd_touch` 框架读取 FT6336U 坐标；
- **LVGL v9**：掌握常用控件（label / button / slider / img）、事件回调（`lv_obj_add_event_cb`）、样式与简单动画；
- **EEZ Studio**：可视化布局、变量（Variables）/ 动作（Action）/ 流程（Flow）机制，以及导出 LVGL 代码后与 ESP-IDF 工程集成；
- **FreeRTOS**：用任务（`xTaskCreate`）管理 UI 刷新与控制逻辑，避免阻塞式延时。

基础要求：

1. 根据屏幕型号和通信接口正确接线，使 ESP32-S3 稳定驱动 TFT 屏幕（参考工程：ST7789V 240×320 SPI + FT6336U 触摸 I2C）；
2. 屏幕能正确显示文字、图形、图片，触摸点击响应准确（无漂移）；
3. 用 **EEZ Studio v0.29.0** 可视化绘制界面并**导出 LVGL v9 代码**，集成进 ESP-IDF 工程（组件依赖：`lvgl/lvgl 9.2.0` + `espressif/esp_lvgl_port 2.8.0` + `espressif/esp_lcd_touch_ft5x06 1.1.0`）；

学习参考：

[esp32 lvgl9.x EEZ Studio 教程（Squareline Studio 平替，含软件下载与安装）](#)

4. 在 README 中说明屏幕型号、引脚连接、所用图形库 / 工具版本、功能设计和使用方法，并附上能展示实际运行效果的图片或视频。

建议方向（推荐）：

- **屏幕控制灯**：触摸屏上做灯状态页——开关按钮、亮度滑条、呼吸 / 闪烁模式切换，与 TASK1~TASK4 的灯控逻辑联动；
- **屏幕控制电机**：触摸屏上做电机页——启停、正反转、速度滑条，进阶可实时显示编码器测得的转速（与 TASK5 联动）；
- **状态显示**：联网页显示 Wi-Fi 连接状态与 IP（与 TASK4 联动）；
- 理解 EEZ Studio 的 **变量（Variables）/ Action（动作）/ Flow（流程）** 机制：`action.cpp` / `vars.cpp` 为手动编写的用户代码，EEZ 导出时不会被覆盖，UI 与业务逻辑通过它们衔接（参考工程结构）。

本任务不以界面复杂程度作为唯一评价标准。创意的完整性、实际运行效果、对实现原理的理解，以及解决调试问题的过程同样重要。

提交要求

一、提交内容

1. 提交完整、可构建和可运行的 ESP-IDF 工程（不要只提交单个 `.c` 文件）；上传到你的 GitHub 仓库。
2. 在 README 中写明：开发环境（含 **ESP-IDF 与各组件的准确版本号**）、依赖、构建与烧录命令、引脚连接、串口 / 网络 / BLE 指令格式，以及各任务完成情况。
3. 使用截图或视频展示点灯、呼吸灯、串口 / Wi-Fi/BLE 控制、电机调速与闭环效果、触摸屏界面的实际运行效果。
4. 说明主要设计思路、关键参数（PWM 频率 / 分辨率、PID 参数、编码器分辨率）、调试过程及尚未解决的问题。即使部分功能未完成，也应保留有效尝试和分析。

二、提交格式（工程目录结构，参考考核组参考工程 `dough_mixer` 的组件化风格）

工程按**外设 / 功能模块**拆分源码目录（`components/BSP/` 下每个外设一个文件夹，总线与器件分开），**BSP 整体作为单个组件**，由一个 `CMakeLists.txt` 统一注册所有子目录（同 `dough_mixer`），禁止把所有代码堆在 `main` 里：

```
1  <你的项目名>/
2  |— main/
3  |   |— main.c           # 应用入口，初始化所有外设
4  |   |— CMakeLists.txt
5  |   |— idf_component.yml # 组件依赖（lvgl/lvgl 9.2.0 等，见环境要求）
6  |— components/BSP/     # BSP 为单个组件，由一个 CMakeLists.txt 统一注册
7  |   |— CMakeLists.txt  # idf_component_register(SRC_DIRS ...) 注册下面
   |   所有子目录
8  |   |— GPIO/           # TASK1 GPIO 点灯
9  |   |   |— led.c       # 点灯逻辑
10 |   |   |— led.h
11 |   |— LEDC/           # TASK2 LEDC 呼吸灯（PWM）
12 |   |   |— breath.c    # 呼吸灯渐变逻辑
13 |   |   |— breath.h
14 |   |— UART/           # TASK3 串口指令解析（中断接收 + 环形缓冲）
15 |   |   |— uart_cmd.c  # 串口驱动 + 指令解析
16 |   |   |— uart_cmd.h
17 |   |— WIFI/           # TASK4 Wi-Fi 连接
18 |   |   |— wifi_ap.c   # Wi-Fi AP / STA 初始化
19 |   |   |— wifi_ap.h
20 |   |— HTTP/           # TASK4 esp_http_server + 内嵌网页
21 |   |   |— http_server.c # HTTP 服务器 + 网页
22 |   |   |— http_server.h
23 |   |— BLE/            # TASK4 蓝牙（外接 HC-04D 模块，UART 透传）
24 |   |   |— bt_module.c # HC-04D AT 配置 + 数据透传
25 |   |   |— bt_module.h
26 |   |— MOTOR/          # TASK5 电机驱动（LEDC PWM 调速 + 方向控制）
27 |   |   |— motor.c     # 电机启停 / 调速 / 方向
28 |   |   |— motor.h
29 |   |— ENCODER/        # TASK5 拓展 编码器测速（PCNT）
30 |   |   |— encoder.c   # PCNT 配置 + 转速换算
31 |   |   |— encoder.h
32 |   |— PID/            # TASK5 拓展 PID 速度闭环
33 |   |   |— pid.c       # PID 计算（位置式 / 增量式）
34 |   |   |— pid.h
35 |   |— SPI/            # SPI 总线（LCD 专用）
36 |   |   |— spi.c       # SPI2 总线初始化
37 |   |   |— spi.h
38 |   |— ST7789V_LCD/    # TASK6 LCD 面板驱动
39 |   |   |— lcd.c       # ST7789V 初始化 + 绘图
40 |   |   |— lcd.h
41 |   |— IIC/            # I2C 总线（触摸专用）
42 |   |   |— iic.c       # I2C 总线初始化
43 |   |   |— iic.h
44 |   |— FT6336U_TOUCH/  # TASK6 触摸驱动（兼容 FT5x06）
45 |   |   |— touch.c     # 触摸初始化 + 坐标读取
46 |   |   |— touch.h
47 |   |— LVGL_UI/        # TASK6 LVGL 显示 + 触摸注册
48 |   |   |— lvgl_ui.c   # LVGL 初始化 + 输入设备注册
```

```
49 | | └─ lvgl_ui.h
50 | └─ ui/ # TASK6 EEZ Studio 生成 + 用户编写
51 | └─ action.cpp # Action 实现（不被 EEZ 覆盖）
52 | └─ vars.cpp # 变量实现（不被 EEZ 覆盖）
53 | └─ ... # 其余 EEZ 导出文件（ui.c / screens 等）
54 └─ lv_conf.h # LVGL v9 配置
55 └─ partitions.csv # 分区表（16MB Flash，参考 dough_mixer）
56 └─ sdkconfig # ESP-IDF 配置
57 └─ CMakeLists.txt
58 └─ README.md
```

三、README 格式

章节	内容要求
项目简介	一句话说明 + 技术栈（ESP32-S3 + LVGL v9 + EEZ Studio 等）
硬件引脚	表格：信号 / GPIO / 方向 / 说明，按外设分组（LCD、触摸、电机...）
关键参数	PWM 频率与分辨率、编码器线数与倍频、PID 参数（P/I 值）、串口波特率等
控制逻辑	串口 / BLE 指令格式表、HTTP API（若做 Wi-Fi）
项目结构	目录树（同 "二"）
构建与烧录	idf.py set-target esp32s3、idf.py build flash monitor 等命令
版本号	ESP-IDF 与各组件的准确版本号（以命题机统一环境为准，见上）
任务完成情况	各 TASK 完成度 + 运行效果截图 / 视频
常见问题	调试中遇到的问题与解决方式

说明：组件目录命名不强求与参考工程完全一致，但
必须按外设 / 功能拆分
，体现模块化与可移植性，这也是评分的参考点之一。

参考资料

1. [ESP-IDF 编程指南（官方中文文档）](#)
2. [ESP-IDF 环境搭建](#)
3. [ESP-IDF 组件注册表 \(lvgl/esp_lvgl_port/esp_lcd_touch 版本查询\)](#)
4. [LVGL v9 文档](#)
5. [EEZ Studio 官方文档 · EEZ Studio 下载 \(v0.29.0\)](#)
6. [EMQX（本地 MQTT Broker，本期不做要求，仅了解）](#)
7. [【正点原子】ESP32 快速入门开发秘籍（IDF 版）【真人出镜】](#) —— 环境搭建与入门
8. [【PID 入门教程 - 电机控制 倒立摆 全程手把手打代码调试】](#) —— TASK5 PID 闭环
9. [esp32 lvgl9.x EEZ Studio 教程（Squareline Studio 平替）](#) —— TASK6 EEZ Studio 上手
10. [8 分钟搞定！Claude Code 保姆级安装 + 原理 + 真实用法（国内直连）](#) —— AI 编程工具（注意考核 AI 政策：可用 AI 辅助，但代码必须自己理解，不得完全依赖 AI 生成）

11. [给傻子的 Git 教程](#) —— GitHub 提交入门
12. [电机控制进阶 ——PID 速度控制 \(CSDN\)](#) —— (位置式 PID 与参数整定实验)
13. [PID 算法讲解 + PID 电机闭环控制介绍 \(CSDN\)](#) —— (PWM 与 PID 联系、调参经验)

四、机器人仿真与视觉

任务目标

配置 Ubuntu 与 ROS 2 开发环境。

了解 ROS 2 的基本工作方式，掌握工作空间构建、节点运行、Topic 通信、TF 坐标变换和常用调试工具；在 Gazebo 中完成移动机器人与仿真环境的设计，使用激光雷达进行 SLAM 建图，并利用传感器数据实现简单的自主探索和视觉目标跟随。

本部分提供一个可运行的差速移动机器人仿真起始工程，位置为：

```
1 | part3/ROS2-Recruitment-Starter-main/recruit_robot_sim
```

起始工程已经提供机器人基本模型、Gazebo 场景、差速驱动、激光雷达、RGB 相机、里程计、TF、RViz 和 SLAM Toolbox 配置，但**不包含**自主探索、颜色跟随、路径规划或现成的自主导航程序。你需要阅读工程和相关文档，在此基础上独立完成以下任务。

环境要求

- Ubuntu 22.04
- ROS 2 Humble
- Gazebo Classic 11 与 `gazebo_ros_pkgs`
- `slam_toolbox`、`teleop_twist_keyboard`
- OpenCV 与 `cv_bridge`

Windows 用户推荐使用 **WSL2 + Ubuntu 22.04** 配置环境；也可以使用原生 Ubuntu 22.04。请勿使用 WSL1。使用 Windows 11 时可通过 WSLg 运行 Gazebo、RViz 等 Linux 图形程序，并建议先将 WSL 更新至最新版本。如果当前设备无法正常运行 WSLg，或 Gazebo 图形界面存在严重兼容性问题，可以改用原生 Ubuntu 环境。

wsl2+ubuntu安装参考资料：

blog.csdn.net/Natsuago/article/details/145594631

使用 WSL2 时，建议将 ROS 2 工作空间放在 WSL 的 Linux 文件系统中，例如 `~/ros2_ws`，不要直接放在 `/mnt/c/`、`/mnt/d` 等 Windows 挂载目录中，以免编译和文件访问速度受到影响。请为 WSL2 预留足够的内存和磁盘空间，并确认显卡驱动支持 WSLg 图形加速。

依赖安装、工程构建、启动方式及 Topic 说明请先阅读 `ros/ROS2-Recruitment-Starter-main/recruit_robot_sim/README.md`。完成构建后，可分别使用以下命令启动基础仿真或 SLAM：

```
1 | ros2 launch recruit_robot_sim bringup.launch.py
2 | ros2 launch recruit_robot_sim slam.launch.py
```

任务介绍 (推荐阅读英文原文 part3/ROS2-Recruitment-Starter-main/recruit_robot_sim/README.md)

• TASK1: 配置环境、运行并理解基础工程

首先完成 Ubuntu 22.04、ROS 2 Humble、Gazebo Classic 11 及本题所需依赖的安装。

建议自行学习并配置 WSL2，理解 Windows、WSL2 和 Ubuntu 三者之间的关系，以及如何进入 Linux 终端、访问文件和运行图形程序。

按照起始工程 `ros/ROS2-Recruitment-Starter-main/recruit_robot_sim/README.md` 的说明创建 ROS 2 工作空间、放置功能包并使用 `colcon build` 完成编译。

正确加载 ROS 2 和工作空间环境后，启动 Gazebo 与 RViz，确认机器人模型、激光雷达数据和相机画面均能正常显示。不得仅展示安装完成页面，应以工程实际成功运行作为环境配置完成的标准。

成功启动起始工程后，使用 `teleop_twist_keyboard` 控制机器人运动。在 RViz 或 ROS 2 命令行工具中查看并理解 `/cmd_vel`、`/scan`、`/odom`、`/camera/image_raw` 和 `/map` 等 Topic 的数据及作用。

观察机器人的 TF 树，理解 `map → odom → base_footprint → base_link` 以及传感器坐标系之间的关系。在 README 中记录采用的环境方案、关键安装与构建步骤、软件版本、使用过的检查命令和遇到的问题，并展示 Gazebo、RViz、激光雷达和相机均正常工作的结果。

• TASK2: 设计机器人与仿真环境

修改提供的 Xacro、URDF 和 World 文件，设计属于自己的差速移动机器人及仿真场景。

可以调整机器人尺寸、外观、传感器安装位置、场地结构、障碍物类型与分布，也可以创建或导入自己的模型。

最终提交的机器人和场景不得保持起始工程原样。修改后应保证模型能够正常生成、轮式运动与传感器数据正常，且场景适合后续建图、探索和颜色跟随任务。

• TASK3: 手动控制与 SLAM 建图

启动 SLAM Toolbox，通过键盘控制机器人遍历自己设计的环境，完成一幅较完整的二维栅格地图并保存建图结果。

建图过程中应尽量覆盖可通行区域，避免碰撞，并观察激光点云、机器人位姿、地图更新及回环修正。提交最终地图及建图过程的截图或视频，并简要说明遇到的问题和解决方法。

• TASK4: SLAM 参数调节与对比

在成功完成基础建图后，根据自己的机器人和环境修改 `config/slam_params.yaml`。你可以调节地图分辨率、激光雷达有效范围、地图更新阈值、扫描匹配、回环检测或其他相关参数。

至少完成一组调参前后的对比实验，保留两次建图结果，并说明修改了哪些参数、修改理由及其对地图质量和运行效果的影响。评价时更重视实验过程和分析，不以单纯堆叠参数数量作为标准。

• TASK5: 基于激光雷达的自主探索建图

自行编写 `exploration.py` ROS 2 节点，订阅 `/scan`，并向 `/cmd_vel` 发布 `geometry_msgs/msg/Twist`，使机器人能够在未知环境中自主移动、避障并完成建图，不再依赖键盘持续控制。

基础实现可以比较前、左、右方向的测距结果，在接近障碍物时停止、后退或选择转向。有余力可进一步实现多区域感知、脱困策略、随机转向、速度控制、基于里程计的卡死检测、已访问区域记录或探索完成条件。程序退出或发生异常时必须主动发布零速度指令，使机器人安全停止。

- **TASK6：基于 OpenCV 的颜色目标跟随**

自行编写 `color_follow.py` ROS 2 节点，订阅 `/camera/image_raw`，使用 `cv_bridge` 将 ROS 图像转换为 OpenCV 图像，并识别场景中的红色、绿色或蓝色目标。

建议使用 HSV 阈值分割、形态学处理和轮廓分析获得目标位置与大小，再根据目标中心相对图像中心的偏差控制机器人转向。目标面积较大时应减速或停止；目标丢失时应停车或以较低速度搜索。程序应能应对一定程度的图像噪声，不得仅针对某一帧或固定坐标编写控制逻辑。

- **TASK7（附加题）：基于 Nav2 的自主导航**

使用 TASK3 或 TASK5 建立并保存的二维栅格地图，配置 Nav2 导航系统，使机器人能够在已知地图中完成自主定位、全局路径规划、局部避障和运动控制。

首先保存 SLAM 建图生成的 `.pgm` 和 `.yaml` 地图文件，并使用 Map Server 加载地图。使用 AMCL 或 SLAM Toolbox 定位模式确定机器人在地图中的位姿，在 RViz 中设置初始位姿后，通过 `Nav2 Goal` 或 `2D Goal Pose` 发布导航目标。

Nav2 参数应根据 TASK2 中自行设计的机器人尺寸、运动能力和仿真场景进行配置，不得直接照搬起始工程中的全部默认参数。应重点检查：

1. 机器人的 `footprint` 或 `robot_radius`；
2. 最大线速度、最大角速度及加减速度限制；
3. 全局代价地图与局部代价地图的尺寸、分辨率和更新频率；
4. 障碍图层、静态地图层和膨胀层；
5. `inflation_radius` 与 `cost_scaling_factor`；
6. 全局规划器、局部控制器及其主要参数；
7. 目标位置与目标姿态容差；
8. 卡死、碰撞风险或规划失败时的恢复行为。

至少设置三个具有不同位置和朝向的导航目标，使机器人在不依赖键盘持续控制的情况下依次到达目标点。导航路线中应至少包含一次明显转向，并经过一个障碍物附近或较窄的可通行区域。机器人在导航过程中应能够生成全局路径，根据局部环境调整运动，并避免与场景中的静态障碍物发生碰撞。

在 RViz 中展示地图、机器人定位结果、全局路径、局部轨迹以及全局和局部代价地图，并提交完整导航过程的截图或视频。在 README 中说明全局规划器、局部控制器、全局代价地图和局部代价地图各自承担的作用。

[Nav2入门教程](#)，学习后可自行查找开源nav2解决。

本任务为附加题。评价时重点考查 Nav2 配置与机器人模型的匹配程度、导航过程的完整性、参数分析和问题排查能力。

提交要求

1. 提交完整、可构建和可运行的 ROS 2 工程，不要只提交单个脚本。
2. 在 README 中写明系统环境、依赖安装、构建与运行命令，以及各任务的完成情况。
3. 提交自定义机器人和场景、保存的地图、SLAM 调参前后对比，以及自主探索和颜色跟随的代码。
4. 使用截图或视频展示 Gazebo、RViz、建图、自主探索和颜色跟随的实际运行效果。
5. 说明主要设计思路、关键参数、调试过程及尚未解决的问题。即使部分功能未完成，也应保留有效尝试和分析。

任务注解

- 建议按 TASK1 至 TASK6 的顺序完成，先确认 Topic、TF 和传感器数据正确，再编写控制算法。
- 可以使用现有 ROS 2、SLAM 和 OpenCV 库，但必须理解自己使用的接口及主要参数，能够解释节点之间的数据流。
- 自主探索和颜色跟随允许采用简单但可靠的方法；功能完整、运行稳定、分析清晰比盲目追求复杂算法更重要。

五、电路基础

to be continued.....(by wlt)

六、最终说明

你需要在**10月6号**之前上传你完成的全部内容（即使你没有做完，也必须提交）。

同时请确保已提交有效报名表，否则视为**未参加考核**。

第一阶段完成的任务文件

位置：第一阶段总指导书同级目录下

文件：Summary_Report1.pdf + 所有文件

格式要求：格式自拟

需要描述以下内容：

1. **开头描述：**做了哪些 lab，做到什么程度。
2. **运行结果截图：**做了的所有 lab 的每一个运行结果截图，并且在图下说明是哪个 lab 的哪个部分。
3. **C 相关的 lab：**要把 .exe 放进去，其他的中间产物可以不管。

第二阶段完成的任务文件

位置：第二阶段总指导书同级目录下

文件：Summary_Report2.pdf + 所有文件

格式要求：格式自拟，但是不得多于两页

需要描述以下内容：

1. **选择的题目：**比方说，选择 part1 and part3。
2. **完成的进度：**比方说，完成 part2 的前三个 task，或者 part1 到某一个部分。

上传要求：最后两个阶段的任务总共两个文件夹上传到你的 GitHub 仓库。

出题人: 李昊轩 薛子婧 田子禾 王良涛 赖嘉和

审题人: 李昊轩 薛子婧 田子禾 王良涛 赖嘉和

祝各位同学考核顺利，收获成长！

所有答题文件